STM32 示波器显示端 · 学习路线与知识详解

> 面向本项目（STM32F746NGH6 + TouchGFX + USART6 数据链路）的学习笔记。 每个知识点都标注了「为什么需要 / 项目中的例子 / 学到什么程度」，最后附学习路线与自检表。

---

0. 怎么用这份文档

- 按章节顺序读，每章独立成篇，可以跳读。
- 每章末尾有「自检」：能不看代码回答，才算真懂。
- 代码引用都以本工程实际文件为准，路径相对工程根目录。

---

1. 项目全景

1.1 硬件

项目	参数
MCU	STM32F746NGH6，Cortex-M7，主频 108 MHz
显示	480×272 RGB565，LTDC + SDRAM 帧缓冲
触摸	FT5336（I2C，坐标需交换 X/Y）
数据链路	USART6，PC6=TX / PC7=RX，921600 8N1，DMA 循环接收
导出链路	USART1（ST-Link 虚拟串口），115200 / 460800

1.2 软件分层

```
采集端(另一块板/PC)
      │  1706 字节数据帧 @20fps
      ▼
USART6 + DMA2 Stream1（循环接收 4096B 环形缓冲）
      ▼
Link.hpp          帧同步 → 校验 → 解包 → 写入 ScopeData
      ▼
ScopeData         500 点/通道 + 触发/测量参数 + 链路状态
      ▼
Model::tick()     60Hz：取帧、触发判断、measure()、refreshCount++
      ▼
screenView        检测 refreshCount 变化 → invalidate 局部区域
      ▼
ScopeWidget       按 invalidatedArea 裁剪绘制波形/面板/数字通道
      ▼
LTDC → LCD


下行使能/时基/触发命令：8 字节命令帧，USART6 发回采集端
```

1.3 关键数字（要记住，联调时全靠它）

参数	值	说明
每屏采样点	500	10 格 × 每格 50 点
每格像素宽度	34 px	波形区宽 340 px
帧长	1706 B	12 头 + 1692 载荷 + 2 CRC
载荷	1692 B	CH1 750 + CH2 750 + D0/D1/D2 各 64
帧率	20 fps（可调）	每帧只传"一屏窗口"
带宽占用	1706×20 ≈ 34.1 kB/s	921600 8N1 下约 37%
单帧发送耗时	≈18.5 ms	921600 下
界面节拍	60 Hz	每 tick 16.7 ms 预算

时基与等效采样率：

时基	一屏时间	采样间隔	等效采样率
10 ms/div	100 ms	200 µs	5 kSa/s
5 ms/div	50 ms	100 µs	10 kSa/s
1 ms/div	10 ms	20 µs	50 kSa/s

时基	一屏时间	采样间隔	等效采样率
500 µs/div	5 ms	10 µs	100 kSa/s
100 µs/div	1 ms	2 µs	500 kSa/s

> 注意：500 kSa/s 是"窗口内的等效采样率"，不是持续数据流。每帧只抓 1 ms 的窗口，所以带宽不随时基变化。

---

2. 嵌入式 C 基础

2.1 指针与数组

- 数组名在表达式中退化为首元素指针，`sizeof(arr)` 只在定义处有效，传参后 `sizeof` 变成 4（指针大小）。
- 函数传数组必须同时传长度，这是 C 的约定；本项目 `drawAnalog(const int16_t* samples, ..., int count)` 就是这么做的。
- 指针算术以元素为单位：`p+1` 前进 `sizeof(*p)` 字节。

常见坑：`int16_t ch1[500]` 的 `ch1[i]` 在 `i=500` 时不会报错，只会踩到相邻内存。协议里 payloadLen 必须校验，本质就是防止越界。

2.2 结构体与内存布局

- 结构体成员按声明顺序排列，编译器会插入填充字节（padding）来满足对齐。跨设备/跨语言传结构体时不能直接 `memcpy`，必须逐字段序列化——这也是为什么协议里每个字段都显式规定字节序和长度。
- 位域（`unsigned a:4;`）的布局依赖编译器实现，不建议用于协议；协议里用"位打包 + 移位"更可控。
- 联合体（union）可用来做同一块内存的多视图，但同样有字节序问题。

2.3 volatile / static / const

- `volatile`：告诉编译器"这个值可能被外部改变，不要缓存到寄存器"。DMA/中断写的缓冲区、硬件寄存器必须是 volatile 或通过访问函数读写。
- `static`：函数内 `static` 变量生命周期贯穿程序；文件作用域 `static` 表示内部链接（仅本文件可见）。
- `const`：只读语义，配合 `const int16_t*` 让编译器知道函数不会改数据，也便于优化。

`Link.hpp` 的 `rxBuf` 由 DMA 写、CPU 读，属于典型的"共享内存"，读取时机与缓存一致性都要注意（见 3.7）。

2.4 位运算

四个基本操作，必须条件反射式写出来：

```
x |=  (1u << n);      // 置位
x &= ~(1u << n);      // 清位
x ^=  (1u << n);      // 翻转
if (x & (1u << n))    // 测试
```

本项目里： - 通道使能掩码：bit0=CH1、bit1=CH2、bit2..4=D0..D2。 - 触发命令打包：`src | (edge<<3) | (mode<<5)`，一个字节塞三样东西。 - 数字通道位打包：第 i 个采样点放在第 `i>>3` 字节的第 `i&7` 位。

2.5 字节序与 12bit 打包

STM32 是小端：多字节整数低字节在前。协议里 `len` 和 `tick_ms` 都是小端。

12bit ADC 值的打包是工程里最值得反复看的一段：

```
// 两个 12bit 采样 -> 3 字节（打包端，Python 版）
out[0] = s0 & 0xFF;
out[1] = ((s0 >> 8) & 0x0F) | ((s1 & 0x0F) << 4);
out[2] = (s1 >> 4) & 0xFF;

// 解包端（C++ 版）
s0 = (uint16_t)(b[0] | ((b[1] & 0x0Fu) << 8));
s1 = (uint16_t)((b[1] >> 4) | (b[2] << 4));
```

500 点 × 12 bit = 6000 bit = 750 字节，正好。**必须先想清楚位图，再写代码**，这类代码靠试是试不对的。

2.6 内存分区

区域	内容	本项目
.text	代码、常量	Flash
.data	已初始化全局/静态变量	启动时从 Flash 拷贝到 RAM
.bss	未初始化变量	启动时清零
栈	局部变量、函数调用	在 startup 文件里设置大小
堆	malloc	本项目基本不用
SDRAM	大块缓冲	帧缓冲在 `0xC0000000`

嵌入式里"栈溢出"是最隐蔽的 bug：栈太小、递归、大数组放局部变量都会踩。本项目把 `rxBuf[4096]`、`frame[1714]` 放在类成员里（静态区）而不是局部变量，就是这个原因。

2.7 常见坑清单

1. 有符号右移、`char` 是否有符号（ARM 上 `char` 默认无符号）。
2. 隐式整型提升：`uint8_t a=200, b=100; (a+b)` 结果是 300，但赋回 `uint8_t` 会截断。
3. 位运算优先级低于比较：`if (x & mask == 0)` 是错的，要写 `if ((x & mask) == 0)`。

4. `memcpy` 结构体跨编译器不可靠。
5. 中断里调用耗时函数或 `printf`。
6. 浮点在 M7 上可用但有代价；波形绘制尽量用整数运算。

2.8 自检

- 能解释 `b[1] >> 4` 为什么取的是 s1 的低 8 位。
- 能说出 `volatile` 缺失时可能出现的现象。
- 能算出 500 点 12bit 数据的字节数，并解释为什么是 750 而不是 1000。

---

3. STM32F746 外设与 HAL

3.1 时钟树

`SystemClock_Config()` 大致路径：HSE（外部晶振）→ PLL → SYSCLK 108 MHz → AHB/APB 分频。

要建立的直觉： - 外设时钟来自哪条总线（APB1/APB2），决定它的最大频率和波特率精度。 - USART6 挂在 APB2，时钟 108 MHz。 - 108 MHz / 921600 = 117.1875，小数部分 0.1875 = 3/16，**刚好能被 16 倍过采样精确表示**，所以 921600 在这个主频下没有波特率误差。这是个很值得记住的巧合；换成 115200 反而要注意误差。

3.2 GPIO 复用

一个引脚要当串口用，需要三步： 1. 开 GPIOC 时钟； 2. `MODER` 设为复用模式（10）； 3. `AFR` 选 AF8（USART6）。

`Link::init()` 里用 HAL 的 `HAL_GPIO_Init` 完成，配置项 `Alternate = GPIO_AF8_USART6`、`Pull = GPIO_PULLUP`。

3.3 UART 关键参数

- 921600 8N1：每字节 10 bit（1 起始 + 8 数据 + 1 停止），所以 921600 bps = 92160 B/s。
- 单帧 1706 B ≈ 18.5 ms，这是"发送一帧要多久"的答案。
- 波特率误差超过约 2%~3% 就会开始出错，所以主频与波特率的整除关系很重要。

3.4 DMA 循环接收

```
hdmaRx.Init.Mode = DMA_CIRCULAR;          // 环形，不停止
hdmaRx.Init.PeripheralInc = DMA_PINC_DISABLE;
hdmaRx.Init.MemInc = DMA_MINC_ENABLE;     // 内存地址自增
hdmaRx.Init.PeripheralDataAlignment = DMA_PDATAALIGN_BYTE;
hdmaRx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
HAL_UART_Receive_DMA(&huart6, rxBuf, RX_SIZE);
```

读数据不需要中断，靠计数器：

```
uint16_t head = RX_SIZE - __HAL_DMA_GET_COUNTER(&hdmaRx);
```

`NDTR` 是"还剩多少没写"，所以"已经写到的位置" = 总长 − NDTR。DMA 写满一圈后自动回到 0，所以 `head` 会回绕，解析器必须能处理"从 4095 跳到 0"。

坑：如果 CPU 读的速度长期追不上写入速度，环形缓冲会被覆盖，表现为"偶发 CRC 错/丢帧"。20fps 下每帧 1706 B，60Hz 轮询一次最多处理约 853 B，所以一次轮询要能把积压全部解析完（`poll()` 里是 while 循环推进）。

3.5 中断与优先级

本项目接收用轮询，不用中断，但要知道： - NVIC 优先级数值越小越高；抢占优先级决定能否打断。 - 中断服务函数要短，只做"搬数据 + 置标志"，解析和绘制放主循环。 - `HAL_GetTick()` 依赖 SysTick，1 ms 精度，本项目用它判断"1 秒内有没有收到帧"。

3.6 `Link::init()` 五步走读

1. `__HAL_RCC_GPIOC_CLK_ENABLE()` / `__HAL_RCC_USART6_CLK_ENABLE()` / `__HAL_RCC_DMA2_CLK_ENABLE()`；
2. PC6/PC7 配成 AF8、上拉、高速；
3. `huart6.Init` 设 921600 / 8N1 / 无流控，`HAL_UART_Init`；
4. `hdmaRx` 配循环模式、字节对齐，`__HAL_LINKDMA(&huart6, hdmarx, hdmaRx)`；
5. `HAL_UART_Receive_DMA(&huart6, rxBuf, RX_SIZE)` 启动。

这套"开时钟 → 配引脚 → 配外设 → 挂 DMA → 启动"的顺序，是 STM32 HAL 的通用套路，换成 SPI/I2C/ADC 也基本一样。

3.7 D-Cache 一致性（F7 特有，必须知道）

Cortex-M7 有 L1 数据缓存。DMA 直接写内存时，CPU 可能读到缓存里的旧数据；反过来 CPU 写的数据也可能还没落到内存。

三种处理方式： 1. 把 DMA 缓冲区放到 MPU 配置为**非缓存/写通**的区域； 2. 读之前 `SCB_InvalidateDCache_by_Addr()`，写之后 `SCB_CleanDCache_by_Addr()`； 3. 数据量小且不追求性能时，直接关 D-Cache（不推荐，帧缓冲性能会掉）。

本工程 `rxBuf` 在普通 RAM，接真实采集端前要确认 MPU 配置；如果出现"数据偶尔不对劲但 CRC 又对"的怪现象，优先怀疑缓存一致性。

3.8 自检

- 能画出"开时钟→配引脚→配外设→挂DMA→启动"的流程。
- 能解释 `RX_SIZE - NDTR` 为什么等于写指针。
- 能说出 921600 下 1706 字节需要多少毫秒。

---

4. 串口帧协议（项目核心）

4.1 数据帧布局（1706 字节）

偏移	长度	字段	说明
0	2	`A5 5A`	帧头
2	1	`01`	版本号（命令帧为 `02`）
3	1	seq	帧序号，0~255 回绕
4	1	timebase	0=10ms 1=5ms 2=1ms 3=500µs 4=100µs
5	1	flags	bit0 CH1、bit1 CH2、bit2..4 D0..D2
6	2	len	载荷长度，小端，固定 1692
8	4	tick_ms	采集端毫秒时间戳，小端
12	1692	payload	CH1 750 + CH2 750 + D0 64 + D1 64 + D2 64
1704	2	crc16	小端，覆盖 `header[2:] + payload`

4.2 载荷细节

- **模拟通道**：500 点 × 12 bit，每 3 字节装 2 点（见 2.5）。数值范围 0~4095，对应 0~3.3 V。
- **数字通道**：500 个 0/1，位打包成 64 字节；第 i 点在 `byte[i>>3]` 的 bit `(i&7)`；后 12 位无效。

4.3 CRC16-CCITT

```
crc = 0xFFFF
for b in data:
    crc ^= b << 8
    for _ in range(8):
        crc = ((crc << 1) ^ 0x1021) & 0xFFFF if (crc & 0x8000) else (crc << 1) & 0xFFFF
```

- 多项式 0x1021，初值 0xFFFF，不反转、不异或输出。
- 覆盖范围必须两端一致：数据帧从 `ver` 到载荷末尾；命令帧从 `0x02` 到 `arg1`。
- 校验范围差一个字节，就会"每帧都错"。

4.4 接收状态机

`Link::parseByte()` 是四状态机：

状态	含义	转移
0	找 A5	收到 A5 → 1，否则留下
1	等 5A	收到 5A → 2；否则回 0（若又是 A5 则停在 1）
2	收 12 字节头	头齐后校验 ver==1、len≤1700；不合法回 0

状态	含义	转移
3	收 1692+2 字节	收齐后校验 CRC；通过则解码

重同步：CRC 失败或长度异常时，不能整段丢掉就等下一帧——如果丢的位置刚好错过帧头，会一直错下去。实现里 `resync()` 把读指针拉到当前 DMA 写位置，从"最新数据"重新找头，这是工程上最省事的做法。

4.5 命令帧（下行，8 字节）

偏移	长度	字段
0	2	`A5 5A`
2	1	`02`（命令版本）
3	1	cmd：1=时基 2=通道使能 3=触发 4=运行
4	1	arg0
5	1	arg1
6	2	crc16（覆盖 `02 cmd arg0 arg1`）

- 时基：arg0 = 0~4。
- 通道使能：arg0 = 位掩码（bit0 CH1、bit1 CH2、bit2..4 D0..D2）。
- 触发：arg0 = `src | (edge<<3) | (mode<<5)`，arg1 = `level>>4`（把 12bit 电平压成 8bit）。
- 运行控制：arg0 = 0 停止 / 1 运行（预留）。

注意：显示端发命令的 8 字节会回到自己的 RX（如果 PC6/PC7 短接），所以解析器必须校验版本号，否则命令帧会被误当作数据帧。

4.6 两端对照

功能	显示端（C++）	采集端参考（Python）
帧同步/解析	`Link::parseByte()`	`parse_commands()`（反向）
组帧	`buildSelfFrame()`	`build_frame()`
12bit 打包/解包	`decodeFrame()`	`pack_analog()`
数字位打包	`decodeFrame()`	`pack_digital()`
CRC	同算法	`crc16()`

两个文件对照着看，是最快理解协议的方法。采集端用哪块板、什么语言都无所谓，只要按这张表实现。

4.7 带宽与采样率

- 每帧 1706 B，20fps → 34120 B/s；921600 8N1 → 92160 B/s，占用 37%。
- 想提高到 50fps：占用 92%，接近极限，且单帧 18.5 ms 已接近 20 ms 周期——不建议。
- 100µs/div 时窗口只有 1 ms，所以"500 kSa/s"不会变成持续流量；真要连续采集，必须换以太网。

4.8 联调 checklist

1. 共地，TX/RX 交叉（显示端 PC6 ← 采集端 TX，PC7 → 采集端 RX）。
2. 两端波特率一致（921600 8N1）。
3. 先发"固定值"帧（CH1 全 2048、数字全 0），确认 LINK 计数增长。
4. 再发正弦，看波形形状与时基是否正确。
5. 最后测命令：改时基，确认采集端收到并改变窗口。
6. 拔掉自测短接线，否则两边 TX 抢线。

4.9 自检

- 能默画 1706 字节布局。
- 能算出 CRC 覆盖范围，并说明命令帧为什么从 `0x02` 开始。
- 能解释 seq 丢帧统计的意义。

---

5. TouchGFX 显示框架

5.1 MVP 三层

层	职责	本项目文件
Model	数据与业务逻辑	`gui/src/model/Model.cpp`
Presenter	转发 View 的操作给 Model	`screenPresenter.cpp`
View	绘制与触摸事件	`screenView.cpp`、`ScopeWidget.hpp`

原则：View 不直接改数据，通过 Presenter 调 Model；Model 不碰 UI。

5.2 tick 时序与 16.7 ms 预算

`Model::tick()` 由框架以 60Hz 调用，一次 tick 的预算约 16.7 ms。里面发生了： 1. 轮询串口（解析积压字节）； 2. 触发判断； 3. `measure()` 扫描 500 点 × 2 通道； 4. 递增 `refreshCount`。

真实教训：自测模式每 50 ms 用 `HAL_UART_Transmit` 阻塞发 1706 字节（18.5 ms），直接把 tick 撑爆 → 触摸响应变慢、偶尔点不动、掉帧。结论：tick 里任何超过几毫秒的阻塞操作都要挪走（改 DMA 发送或降频）。

5.3 invalidate 与局部重绘

- 只有 `invalidate()` 过的区域才会重绘。
- `screenView::handleTickEvent()` 发现 `refreshCount` 变化 → `scope.invalidatePlot()`；读数区约 7Hz 刷一次；标题栏只在状态变化时刷。
- 刷新太频繁会挤占绘制时间，太懒会看起来"卡"。

5.4 单缓冲 + LTDC 条带绘制

本项目用单缓冲 + `REFRESH_STRATEGY_OPTIM_SINGLE_BUFFER_TFT_CTRL`：HAL 按 LTDC 正在扫描的行号，把绘制拆成多条带调用 `draw()`。

后果：**任何绘制函数都必须尊重 `invalidatedArea`**，不能整屏 `memset`，否则会和扫描冲突产生撕裂/花屏。

5.5 裁剪

`ScopeWidget` 里的 `fillRectClip()` 是核心工具：把任意矩形与 `invalidatedArea` 求交再画。

```
int x0 = MAX(rect.x, clip.x);
int x1 = MIN(rect.x + rect.width, clip.x + clip.width);
if (x0 >= x1) return;            // 完全在裁剪区外
```

写图形代码的第一反应应该是"这块会不会画到区域外"。

5.6 header-only 与 .ewp

IAR 工程按 `.ewp` 里的文件清单编译，新增 `.cpp` 要同步改工程文件（容易忘、容易冲突）。所以本项目把控件实现全部写在 `.hpp` 里（header-only），只改代码不改工程结构。

代价：头文件被多处包含时会重复编译，编译时间略长；好处：迁移/备份/协作简单。

5.7 触摸事件

- FT5336 通过 I2C 读坐标，原始坐标与屏幕方向不一致，需要 `x = rawY; y = rawX;`。
- `screenView::handleClickEvent` 只处理 PRESSED/RELEASED，`hitTest()` 把坐标映射到按钮编号。
- 页面用 `panelPage` 状态区分（主面板 / 触发页 / 测量页），命中测试也按页分支。

5.8 常见坑

1. 在 `draw()` 里做耗时计算（每 tick 会被调用多次）。
2. 忘记 `invalidate`，数据变了但屏幕不动。
3. 未裁剪导致区域外花屏。
4. 在 View 里直接改 `ScopeData`，破坏 MVP。

---

6. 波形与测量

6.1 采样值 → 像素

```
x = PLOT_X + i * PLOT_W / (SAMPLES - 1);
y = PLOT_Y + PLOT_H - (value - 0) * PLOT_H / 4095;
```

- 12bit 值 0~4095 映射到波形区高度；`PLOT_H` 通常留一点边距，避免贴边。
- 整数运算即可，注意乘除顺序防止溢出（先乘后除，必要时用 `int32_t`）。

6.2 抽点聚合

500 个采样点、340 像素宽，一列平均 1.47 个点。正确做法是**每列取 min/max 画一条竖线**，而不是隔点取样：

- 隔点取样会漏掉窄脉冲（欠采样显示）；
- min/max 聚合能保证任意窄的尖峰都至少占 1 像素，这是示波器的标准做法。

6.3 触发

模式	行为
AUTO	每帧都刷新；没有触发也画（波形会"跑"）
NORM	只有满足触发条件才刷新，否则保持上一屏
SINGLE	满足一次后冻结

触发条件：模拟通道看"电平穿越"（上升沿：前一点 < level，当前 ≥ level），数字通道只看边沿。`level` 默认 2048（约 1.65 V）。

6.4 锚点对齐与预触发

真实示波器会把触发点固定在屏幕某个横坐标（例如左侧 10%）。做法是把采样窗口循环平移，让触发点落到锚点。

局限：单帧窗口只能循环平移，接缝会出现在最右端，那里可能有一次不连续跳变。

根治：采集端做"预触发采样"——触发点前后各留一半数据再打包发送，显示端就不需要循环平移。这需要改协议语义（`tick_ms` 之外再给触发偏移），属于采集端配合项。

6.5 测量算法

频率（周期法）：

```
periodSamples = secondRise - firstRise;          // 两个上升沿之间隔了多少点
freq = (SAMPLES - 1) / periodSamples * (1 / 一屏时间)
```

实现里为避免浮点，用"每屏点数 / 周期点数"再乘系数，单位是 0.1 Hz。

峰峰值：`(max - min) * 3300 / 4095` mV。

脉宽：上升沿到下降沿的点数 × 采样间隔。

阈值自适应（提高项）：

- 固定 2048 阈值的问题是：信号带直流偏置时（例如 900±800）永远不会穿越 2048，频率显示 0。
- 正确做法：先求 `min/max`，取中点 `(min+max)/2` 作阈值，再用 `1/8 峰峰值` 作迟滞（施密特触发器），避免噪声在阈值附近反复计数。
- 峰峰值过小（例如 < 64 LSB ≈ 52 mV）时判为"无有效信号"，频率显示 0。

6.6 误差来源

1. **量化**：12bit / 500 点，周期测量误差约 ±1 点。
2. **时基精度**：取决于采集端晶振。
3. **抽点**：min/max 聚合会显得波形"粗"，但不会漏峰。
4. **触发抖动**：电平附近有噪声时，触发点会跳 1~2 个点。

6.7 练习

1. 把 CH1 假数据改成 `900 + 800*sin`，观察旧阈值的频率读数变成 0（理解 6.5）。
2. 实现锚点对齐，让边沿固定在第 1 条竖格线上。
3. 把 `measureAnalog` 改成自适应阈值 + 迟滞，验证 50.0 Hz 仍然正确。

---

7. 工具与工程

7.1 IAR

- 工程文件 `EWARM/receive.ewp`；新增源文件必须加入工程，否则不参与编译。
- 命令行编译： `IarBuild.exe EWARM/receive.ewp -build receive -log all`
- 命令行烧录（需先关闭 IAR IDE，否则 ST-Link 被占用）： `cspybat ... --download_only`
- 编译输出看两行： `Total number of errors` / `Total number of warnings`。

7.2 CubeMX

- `.ioc` 是外设配置的唯一源头；改引脚/时钟要在 `.ioc` 里改再生成。
- `Core/Src/*.c` 是生成代码，手改会在下次生成时被覆盖（除非写在 `USER CODE` 区）。
- 本项目 USART6/DMA 是在 `Link.hpp` 里手写初始化的，没有走 CubeMX；这样做的好处是不用改 `.ioc`，坏处是要自己保证时钟/引脚正确。

7.3 Git

- 仓库根目录 `d:\receive - trae`，作者 `王耀臻 <739161958@qq.com>`（仓库级配置）。
- 自动保存： `tools/git-autosave.ps1` 每 20 秒检查一次，有改动就提交 `自动保存 时间戳`；由 `.vscode/tasks.json` 在打开工程时自动启动。
- 正式节点仍建议手动提交并写清 message，自动提交只做兜底。
- 常用： `git log --oneline --graph`、`git show <hash>`、`git diff`、`git revert <hash>`。

7.4 调试手段

手段	用途
串口打印（USART1）	看状态、计数、错误码
逻辑分析仪 / 示波器	看波形、波特率、时序
断点 + 变量窗口	看帧内容、指针位置

手段	用途
`LINK` / `NO LINK` 指示	最快判断链路是否通
导出 BMP	把屏幕内容拿到 PC 上逐像素分析

---

8. 提高项与拓展

项目	要点
暂停/继续	加一个 `paused` 状态，tick 里跳过 refresh
滚动模式	保留环形历史缓冲，每帧左移若干点
回放	保存最近 N 帧到 SDRAM，按时间轴播放
光标测量	两次触摸记录 x，换算时间差与电压差
双通道对比	数学运算（相加/相减）后画第三路
CSV 导出	逐行 `time,ch1,ch2,d0..`，注意串口节流
BMP 导出	54 字节头 + 24bit BGR 行，348×250 约 255 KB
以太网	RMII 引脚、LwIP、TCP 服务端；RAM 预算 30~50 KB
FreeRTOS	通信任务 + 显示任务，注意栈大小与优先级

---

9. 学习路线

阶段	内容	验收标准
1	嵌入式 C（指针/结构体/位运算/字节序）	能独立写出 12bit 打包与解包
2	STM32 外设（时钟/GPIO/UART/DMA）	能不看例程配通一路串口 DMA
3	帧协议（格式/CRC/状态机/重同步）	能默画帧布局并手算 CRC 覆盖范围
4	TouchGFX（MVP/重绘/裁剪）	能新增一个面板且不花屏
5	波形算法（映射/聚合/触发/测量）	能实现锚点对齐与自适应阈值
6	工程化（IAR/CubeMX/Git/调试）	能独立编译、烧录、回滚

推荐顺序：先读 `Link.hpp` + `link_sim.py`（半天能通读），再按 1→2→3→4→5 补基础，最后做提高项。

---

10. 术语表

术语	含义
时基 (timebase)	每格代表的时间，如 1 ms/div
触发 (trigger)	满足条件时开始记录/刷新
预触发	触发点前后各留一半数据的采样方式
迟滞 (hysteresis)	用两个阈值避免噪声反复触发
帧同步	在字节流中定位帧头的过程
重同步	出错后重新寻找帧头
抽点	把多点压缩到一列像素的过程
单缓冲	只有一块帧缓冲，绘制与扫描可能冲突
MVP	Model-View-Presenter，TouchGFX 的架构
NDTR	DMA 剩余传输计数寄存器